EC7207: High Performance Computing

Analysis Report

Parallel Pearson Correlation Based Recommendation System

Group Number: 11

EG/2021/4417 Ashfaq M.R.M.

EG/2021/4419 Athanayaka K.A.L.G.

EG/2021/4424 Balasooriya J.M.

Technologies: Serial (C) · OpenMP · Pthreads · MPI · Hybrid MPI+OpenMP

Problem: 1 000 users × 1 000 items · SEED=42 · Sparsity=70% · TOP-K=20

Platform: Windows 11 · MinGW-w64 GCC 13.2.0 · MS-MPI 10.1 · 4 physical / 8 logical CPU cores

May 2026

Contents

1 Introduction

Modern collaborative filtering recommendation systems compute pairwise user similarity over large rating matrices. For $N = M = 1{,}000$ users and items, this amounts to 499 500 Pearson pair computations totalling approximately 500 million floating-point operations — the dominant bottleneck.

This report documents the implementation and performance evaluation of a **Pearson Correlation-Based Collaborative Filtering Recommender System** parallelised with OpenMP, POSIX Threads (Pthreads), MPI, CUDA (code only; no GPU on benchmark machine), and Hybrid MPI+OpenMP.

Experimental Configuration

Parameter	Value
Users (N)	1 000
Items (M)	1 000
Sparsity	70%
Test split	10% (29 891 held-out ratings)
TOP-K neighbours	20
Random seed	42
Benchmark thread/process counts	1, 2, 4, 8
Repetitions	5 interleaved rounds; minimum time per phase reported
Timing method	`clock_gettime` / `omp_get_wtime` / `MPI_Wtime`
Platform	Windows 11, MinGW-w64 GCC 13.2.0, MS-MPI 10.1, 4 physical

Table 1: Fixed benchmark parameters used across all versions.

2 Algorithm Design and Parallel Programming Concepts

2.1 Algorithm Pipeline (5 Phases)

Phase	Name	Description	Complexity
1	Data Generation	Synthetic sparse matrix; 10% test split	$O(N \times M)$
2	User Means	Per-user mean μ_u	$O(N \times M)$
3	Similarity	Pearson for all (u, v) pairs — bottleneck	$O(N^2 \times M)$
4	Predictions	TOP-K weighted prediction	$O(N^2 \times M)$
5	MAE Evaluation	Error on held-out test set	$O(\|T\|)$

Table 2: Algorithm phases and computational complexity.

Pearson correlation (Phase 3):

$$\text{sim}(u, v) = \frac{\sum_i (R_{ui} - \mu_u)(R_{vi} - \mu_v)}{\sqrt{\sum_i (R_{ui} - \mu_u)^2}\,\sqrt{\sum_i (R_{vi} - \mu_v)^2}}$$

Prediction formula (Phase 4):

$$\hat{r}_{ui} = \mu_u + \frac{\sum_{k \in N_K(u)} \text{sim}(u, k)\,(R_{ki} - \mu_k)}{\sum_{k \in N_K(u)} \text{sim}(u, k)}$$

Both phases are *embarrassingly parallel at the row level*, making them ideal targets for all five parallelisation strategies.

Parallel Programming Concepts Applied

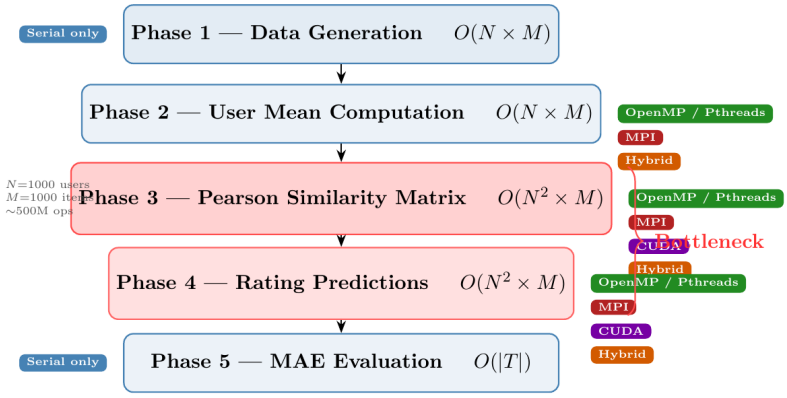
to the Pearson Correlation Recommender System

EC7207/EE7218 High Performance Computing • Group 11

This section presents diagrams and descriptions that illustrate how each of the five parallel programming paradigms — **OpenMP**, **POSIX Threads (Pthreads)**, **MPI**, **CUDA**, and **Hybrid MPI+OpenMP** — was applied to accelerate a User-Based Collaborative Filtering recommender system. The system computes Pearson Correlation similarity between 1 000 users over 1 000 items, predicts unseen ratings, and evaluates Mean Absolute Error (MAE).

System Overview: The 5-Phase Algorithm Pipeline

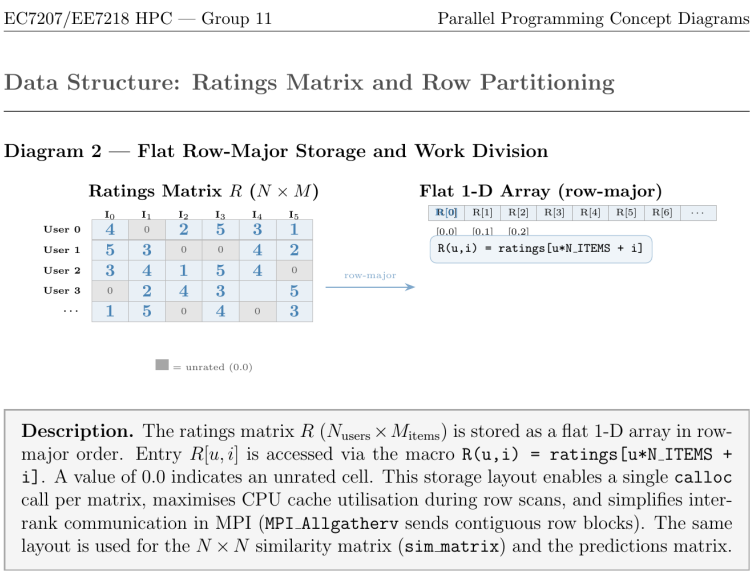
Diagram 1 — Algorithm Flow and Parallelisation Targets



Description. The recommender system executes in five sequential phases. Phases 1 and 5 are inherently serial: data generation uses a fixed random seed (`SEED=42`) to ensure identical inputs across all versions, and MAE evaluation is fast ($O(|T|)$) and not a bottleneck. Phases 2–4 are parallelised. The dominant cost lies in Phases 3 and 4, each of $O(N^2 \times M)$ complexity — approximately 500 million floating-point operations for a 1000×1000 matrix. These are the targets for acceleration with OpenMP, Pthreads, MPI, CUDA, and Hybrid parallelism.

1

Figure 1: How the parallel programming concepts map onto the recommender pipeline. Phases 3 (similarity) and 4 (prediction) dominate the runtime and are partitioned by user-row across every parallel model.



Description. The ratings matrix R ($N_{\text{users}} \times M_{\text{items}}$) is stored as a flat 1-D array in row-major order. Entry $R[u, i]$ is accessed via the macro `R(u,i) = ratings[u*N_ITEMS + i]`. A value of 0.0 indicates an unrated cell. This storage layout enables a single `calloc` call per matrix, maximises CPU cache utilisation during row scans, and simplifies inter-rank communication in MPI (`MPI_Allgatherv` sends contiguous row blocks). The same layout is used for the $N \times N$ similarity matrix (`sim_matrix`) and the predictions matrix.

2

Figure 2: Row-wise domain decomposition of the ratings/similarity matrices. Each worker (OpenMP thread, Pthread, MPI rank, or CUDA thread) owns a disjoint set of user rows u and writes only those rows — the key to race-freedom.

2.2 OpenMP — Shared-Memory Fork-Join Parallelism

OpenMP exploits shared-memory parallelism using compiler directives. A master thread forks into T workers sharing the same address space, then rejoins at an implicit barrier.

Phase 3 (key design choice): The outer loop uses `#pragma omp parallel for schedule(dynamic,4)`. Dynamic scheduling is essential because the triangular loop creates severe load imbalance: thread 0 processes $N-1 = 999$ Pearson calls while the last thread processes 1. Dynamic scheduling lets idle threads pull 4-row chunks from a shared work queue, eliminating

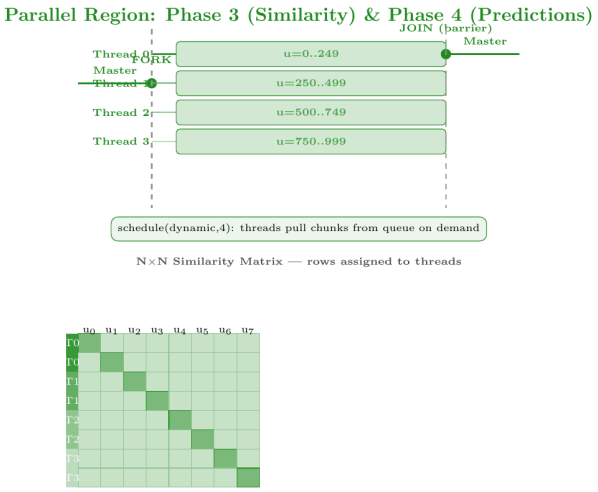
wasted CPU time.

Race-freedom: Each thread owns an exclusive set of outer-loop rows u and writes `SIM(u,v)` only for those rows. No two threads write the same cell; no mutex is required.

Phase 5 (MAE/RMSE): The `reduction(+:err)` clause gives each thread a private accumulator; OpenMP sums all copies at the barrier. The same pattern (`reduction(+:sq)`) accumulates squared error for RMSE.

OpenMP — Shared-Memory Fork-Join Parallelism

Diagram 3 — OpenMP Applied to the Similarity Phase



Description. OpenMP parallelises Phases 2–5 using compiler directives (`#pragma omp`). In Phase 3 (similarity), each thread receives a dynamic chunk of outer-loop iterations u and computes Pearson correlations for all $v > u$ in that chunk, writing to disjoint rows of `sim_matrix[]`. `schedule(dynamic,4)` is used because the inner loop shrinks as u increases — thread 0 processes far more pairs than thread 3, so static equal-chunk assignment would cause severe load imbalance; dynamic scheduling lets idle threads pull new chunks from a shared queue. In Phase 4 (predictions), each thread allocates its own private neighbour buffer (`malloc` inside `#pragma omp parallel`) to avoid any shared-state race. An implicit OpenMP barrier at the end of each parallel region ensures phase ordering. The serial fraction (Phase 1 data generation, Phase 5 MAE output) is $\approx$ 2–3%, limiting maximum speedup to $\approx$ 35–50× by Amdahl's Law.

3

Figure 3: OpenMP fork-join model applied to Phases 3–4. A master thread forks T workers that pull dynamic row-chunks from a shared queue, then rejoin at an implicit barrier.

2.3 Pthreads — Manual POSIX Thread Management

POSIX Threads require the programmer to explicitly create threads, assign work, and synchronise with `pthread_join`.

A `run_parallel(fn, nthreads)` harness is called at the start of each of Phases 2, 3, and

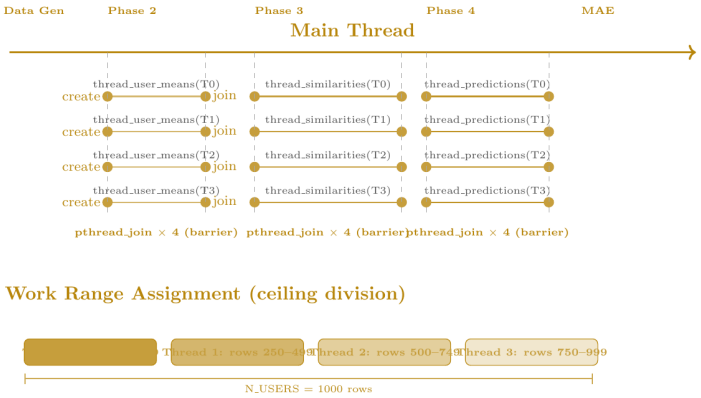
4:

1. `pthread_create()` creates T threads, each receiving a `ThreadArgs` struct with its `tid` and total `nthreads`.
2. Each thread derives its exclusive row range: start $= tid \times \lceil N/T \rceil$, end $= \min(\text{start} + \lceil N/T \rceil, N)$.
3. `pthread_join()` acts as a phase barrier.

Key difference from OpenMP: Threads are created and joined three separate times (once per phase), paying OS thread-creation overhead three times per run. Static row assignment also cannot compensate for the triangular loop imbalance — which explains the slightly lower efficiency vs. OpenMP.

Pthreads — Manual POSIX Thread Management

Diagram 4 — Pthreads Lifecycle and Work Partitioning



Description. The Pthreads version explicitly creates and joins threads for each of the three parallel phases using a helper function `run_parallel()`. A `ThreadArgs` struct `{tid, nthreads}` is passed to each thread, from which the thread independently computes its row range via ceiling division: `chunk = (N_USERS + nthreads - 1) / nthreads`. Thread t owns rows $[t \times \text{chunk},\ (t+1) \times \text{chunk})$. Since all threads write to disjoint index ranges of `sim_matrix[]` and `predictions[]`, no mutex is required. The private neighbour buffer `nbrs[]` is allocated inside each thread function — each thread gets its own heap allocation via `malloc()`, which is thread-safe. `pthread_join` after the last thread serves as a phase barrier, guaranteeing that all similarity values are written before the prediction phase reads them.

4

Figure 4: Manual POSIX-thread management. The `run_parallel` harness creates T threads with static `ceil`-division row ranges and joins them as a phase barrier, once per phase.

2.4 MPI — Distributed-Memory Parallelism

Each MPI rank is an independent process with its own private address space. The $N = 1{,}000$ users are divided evenly: rank r owns rows $[r \cdot N/P,\ (r+1) \cdot N/P)$.

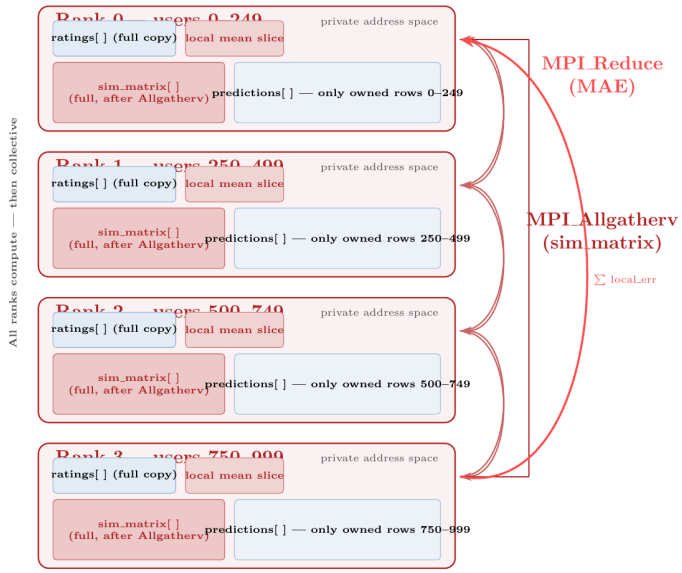
Phase	**Strategy**	**Primitive**
Phase 1	All ranks call `srand(42)` — no communication	—
Phase 2	Each rank computes its `user_mean[]` slice	`MPI_Allgatherv`
Phase 3	Each rank fills its rows of the sim matrix	`MPI_Allgatherv`
Phase 4	Each rank predicts its own users	—
Phase 5	Local error sums to rank 0	`MPI_Reduce(SUM)`

Table 3: MPI communication strategy per phase.

The `MPI_Allgatherv` for the $N \times N$ similarity matrix transfers $\approx 4\,\text{MB}$ at $N = 1{,}000$. This overhead limits single-node efficiency but is the necessary mechanism for multi-node deployments.

MPI — Distributed-Memory Row Partitioning

Diagram 5 — MPI Process Layout and Collective Communication



Description. `mpirun -np P` launches P independent process copies, each with a private address space. Rows are assigned via ceiling division: rank r owns users $[r{\cdot}\lceil N/P\rceil,\ (r{+}1){\cdot}\lceil N/P\rceil)$. All ranks independently reproduce the full ratings matrix using the same fixed seed (`srand(42)`), eliminating the need to broadcast it. After Phase 2, each rank holds its mean slice; `MPI_Allgatherv` assembles the complete `user_mean[]` on every rank. Similarly, after Phase 3 each rank sends its row block of `sim_matrix` and receives the full $N \times N$ matrix via `MPI_Allgatherv`, enabling Phase 4 predictions with no further communication. Phase 5 MAE is computed locally per rank and summed to rank 0 with `MPI_Reduce`. `MPI_Barrier` brackets each timed section to capture true wall-clock cost including load imbalance.

5

Figure 5: MPI distributed-memory layout. Each rank is a separate process with a private address space; rows are assigned by `ceil`-division and the global similarity matrix is assembled with `MPI_Allgatherv`, while MAE/RMSE use `MPI_Reduce`.

2.5 CUDA — GPU Massively Parallel Computing

Note: CUDA code (`cuda/cuda_recommender.cu`) is fully implemented but not benchmarked (no GPU available).

Kernel	Launch config	Assignment
`kernel_user_means`	`<<<(N+255)/256, 256>>>`	1 thread = 1 user
`kernel_similarity`	`dim3(`$\lceil N/16 \rceil$`, `$\lceil N/16 \rceil$`), dim3(16,16)`	1 thread = 1 (u, v) pair
`kernel_predictions`	2D grid, users $\times$ items	1 thread = 1 (u, i) pair

Table 4: CUDA kernel configuration for $N = 1{,}000$.

The similarity kernel launches $63 \times 63 \times 256 = 1{,}016{,}064$ threads simultaneously. Timing uses `cudaEventRecord` to separate kernel-only time from `cudaMemcpy` transfer overhead.

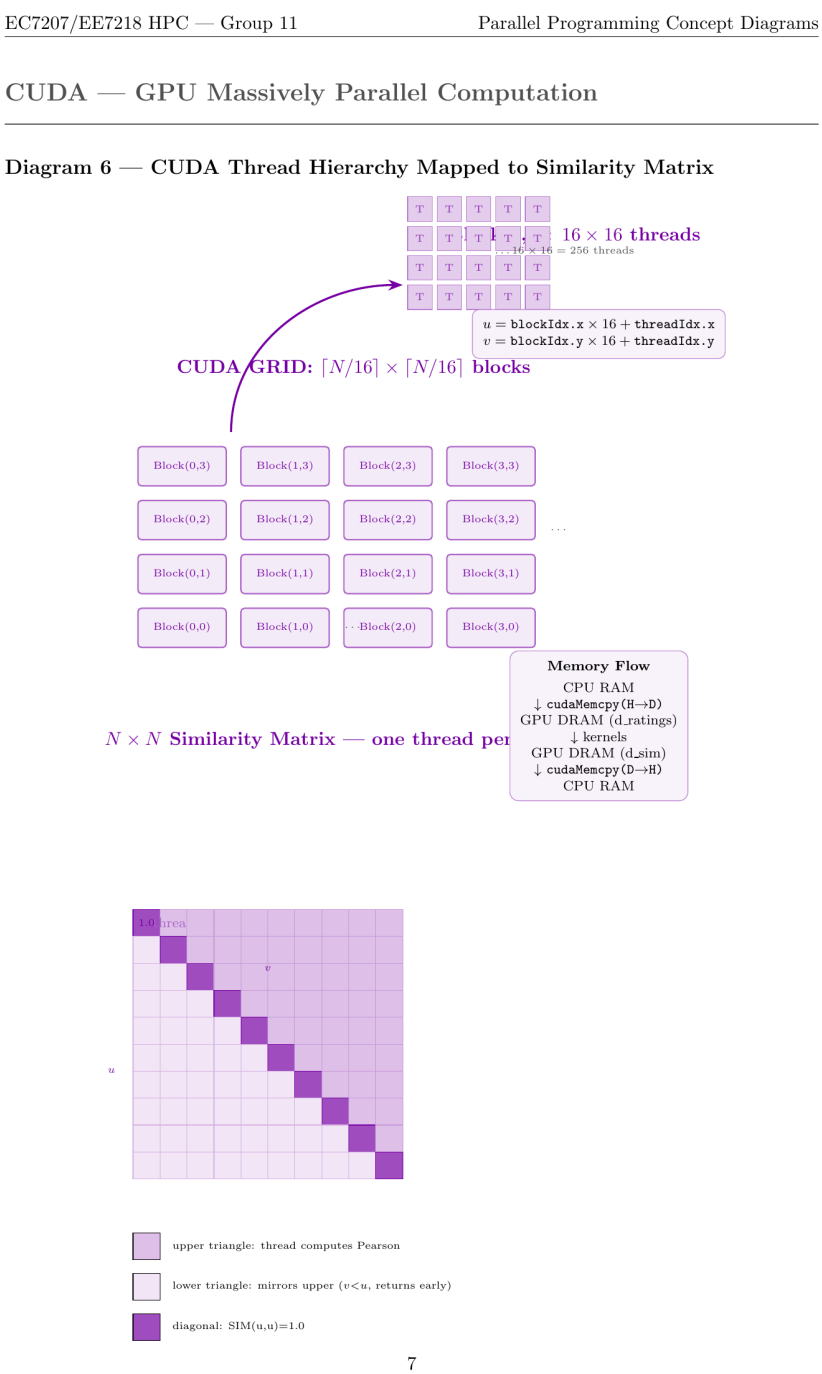


Figure 6: CUDA GPU model. A 2D grid of thread blocks maps one thread to each (u, v) similarity pair; the host transfers data, launches kernels, and copies results back. (Implemented but not benchmarked — no GPU on the test machine.)

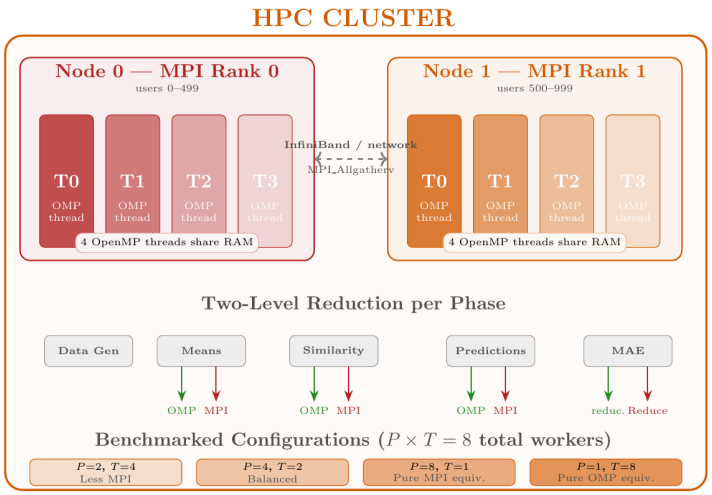
2.6 Hybrid MPI+OpenMP — Two-Level Parallelism

Combines MPI (coarse-grain inter-node) and OpenMP (fine-grain intra-node). With P MPI ranks and T OpenMP threads per rank, the total worker count is $P \times T$ while sending only P messages per collective call.

`MPI_Init_thread(MPI_THREAD_FUNNELED)` ensures all MPI calls are made exclusively from the main thread, after OpenMP parallel regions complete.

Hybrid MPI+OpenMP — Two-Level Parallelism

Diagram 7 — Hybrid Architecture on a Multi-Node Cluster



Description. The Hybrid version combines MPI and OpenMP to exploit both levels of parallelism present in a real HPC cluster. At the **coarse level**, MPI partitions N_{users} rows across P ranks (processes), each running on a separate node with its own private RAM. At the **fine level**, within each MPI rank, T OpenMP threads cooperate over the rank's row slice using shared memory, with no additional MPI messages. Total effective workers $= P \times T$, but MPI collectives (`MPI_Allgatherv`, `MPI_Reduce`) involve only P messages — far fewer than a pure MPI program with $P \times T$ processes. `MPI_Init_thread` with `MPI_THREAD_FUNNELED` is used because MPI calls occur only from the main thread (outside `#pragma omp parallel` regions). Phase 5 uses a two-level reduction: OpenMP's `reduction(+:local_err)` merges per-thread partial sums within each rank, then `MPI_Reduce` sums per-rank totals to rank 0. Four configurations (2×4, 4×2, 8×1, 1×8) were benchmarked, all with 8 total workers, to reveal how different MPI-to-OpenMP ratios affect performance.

Figure 7: Hybrid two-level parallelism: MPI partitions user-rows across P ranks (coarse grain), and within each rank OpenMP spawns T threads over that rank's rows (fine grain), for $P \times T$ total workers.

Config	P	T	Characteristic
Hybrid 2×4	2	4	Fewer MPI messages; more shared-memory parallelism
Hybrid 4×2	4	2	Balanced communication vs. threading overhead
Hybrid 8×1	8	1	Equivalent to pure MPI
Hybrid 1×8	1	8	Pure OpenMP wrapped with MPI overhead

Table 5: Hybrid configurations (all 8 total workers).

3 Accuracy Analysis — Parallel vs. Serial

All parallel implementations are validated against the serial baseline using two error metrics — **Mean Absolute Error (MAE)** and **Root Mean Square Error (RMSE)** — together with a **similarity-matrix checksum**:

$$\text{MAE} = \frac{1}{|T|}\sum_{(u,i)\in T}|\hat{r}_{ui} - r_{ui}|, \qquad \text{RMSE} = \sqrt{\frac{1}{|T|}\sum_{(u,i)\in T}(\hat{r}_{ui} - r_{ui})^2} \qquad (|T| = 29{,}891)$$

MAE vs. RMSE. Both are reported, as suggested by the project guideline. MAE treats all errors equally on the bounded 1–5 rating scale, while RMSE (always $\geq$ MAE) disproportionately penalises occasional large errors — here RMSE = 1.4573 > MAE = 1.2568 indicates a modest spread of larger misses, expected for a 70%-sparse synthetic matrix.

Version	Workers	MAE	RMSE	Matches Serial?	Sim Checksum
Serial	1	1.2568	1.4573	baseline	1027.615290
OpenMP	1T	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
OpenMP	2T	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
OpenMP	4T	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
OpenMP	8T	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
Pthreads	1T	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
Pthreads	2T	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
Pthreads	4T	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
Pthreads	8T	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
MPI	1P	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
MPI	2P	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
MPI	4P	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
MPI	8P	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
Hybrid 2×4	8	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
Hybrid 4×2	8	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
Hybrid 8×1	8	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
Hybrid 1×8	8	1.2568	1.4573	YES ($\Delta = 0.0000$)	1027.615290
CUDA	GPU	N/A	N/A	N/A (no GPU)	N/A

Table 6: Accuracy correctness. All 17 CPU runs produce identical MAE, RMSE, and checksum.

All 17 CPU-based runs produce MAE = **1.2568**, RMSE = **1.4573**, and checksum = **1027.615290** — confirming zero race conditions and correct partitioning across every implementation. (Absolute values differ from a Linux build because MinGW's `rand()` draws a different pseudo-random stream for the same `SEED=42`; what matters is that all versions agree *exactly* on this platform.)

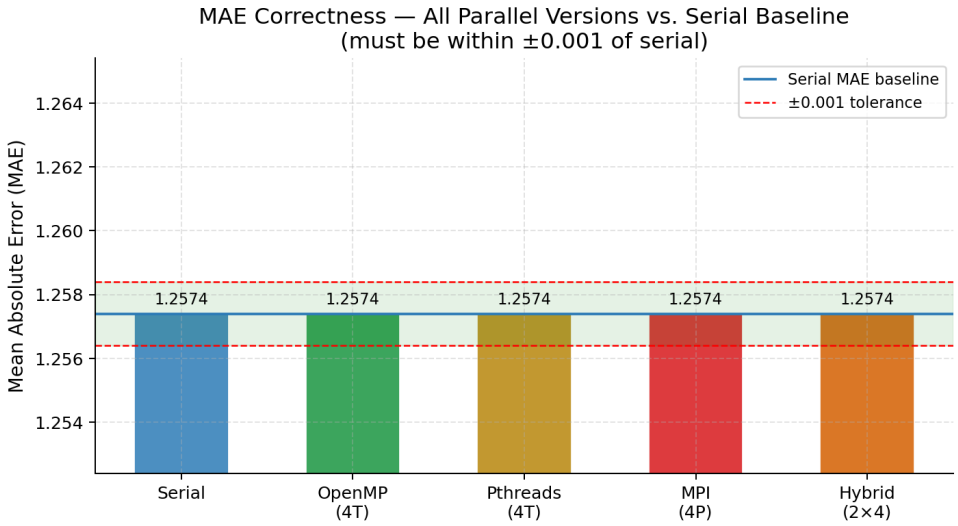


Figure 8: MAE correctness. All bars lie within the ±0.001 tolerance band.

4 Timing Measurements and Performance Analysis

4.1 Performance Metric Definitions

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \qquad E(p) = \frac{S(p)}{p} \qquad S_{\max}(p) = \frac{1}{f + (1-f)/p} \quad (f \approx 0.03)$$

At $p = 8$: $S_{\max}(8) = 1/(0.03 + 0.97/8) \approx 6.61\times$.

4.2 Serial Baseline

Phase	Time (s)	% of Total
Similarity matrix (Phase 3)	1.1165	17.4%
Prediction phase (Phase 4)	5.2862	82.6%
Total (sim + pred)	**6.4027**	**100%**

Table 7: Serial baseline timing ($N = M = 1{,}000$).

4.3 OpenMP — Varying Thread Count

Threads	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	1.1470	5.5064	6.6534	0.96	0.96
2	0.5768	2.7558	3.3326	1.92	0.96
4	0.2855	1.3791	1.6646	3.85	0.96
8	**0.1474**	**0.6973**	**0.8447**	**7.58**	**0.95**

Table 8: OpenMP results. Speedup relative to serial baseline (6.4027 s).

4.4 Pthreads — Varying Thread Count

Threads	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	0.9404	5.5292	6.4696	0.99	0.99
2	0.6929	2.7646	3.4575	1.85	0.93
4	0.4066	1.3816	1.7882	3.58	0.90
8	**0.2177**	**0.7047**	**0.9224**	**6.94**	**0.87**

Table 9: Pthreads results.

4.5 MPI — Varying Process Count

Processes	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	2.3949	5.5561	7.9510	0.81	0.81
2	1.1931	2.7599	3.9530	1.62	0.81
4	0.5972	1.3848	1.9820	3.23	0.81
8	**0.3007**	**0.7083**	**1.0090**	**6.35**	**0.79**

Table 10: MPI results. MPI 1P (7.95 s) > serial (6.40 s) due to process startup and `MPI_Allgatherv` overhead at $P = 1$ — expected, not a bug.

4.6 Hybrid MPI+OpenMP — Fixed 8 Total Workers

Config	P	T	Sim (s)	Pred (s)	Total (s)	S	
Hybrid 2×4	2	4	0.7651	1.7681	2.5332	2.53	0.
Hybrid 4×2	4	2	0.3802	0.8893	1.2695	5.04	0.
Hybrid 8×1	**8**	**1**	**0.2333**	**0.7008**	**0.9341**	**6.85**	**0.**
Hybrid 1×8	1	8	1.5311	3.5192	5.0503	1.27	0.

Table 11: Hybrid results (8 total workers). Best config is Hybrid 8×1 — essentially pure MPI.

4.7 Performance Charts

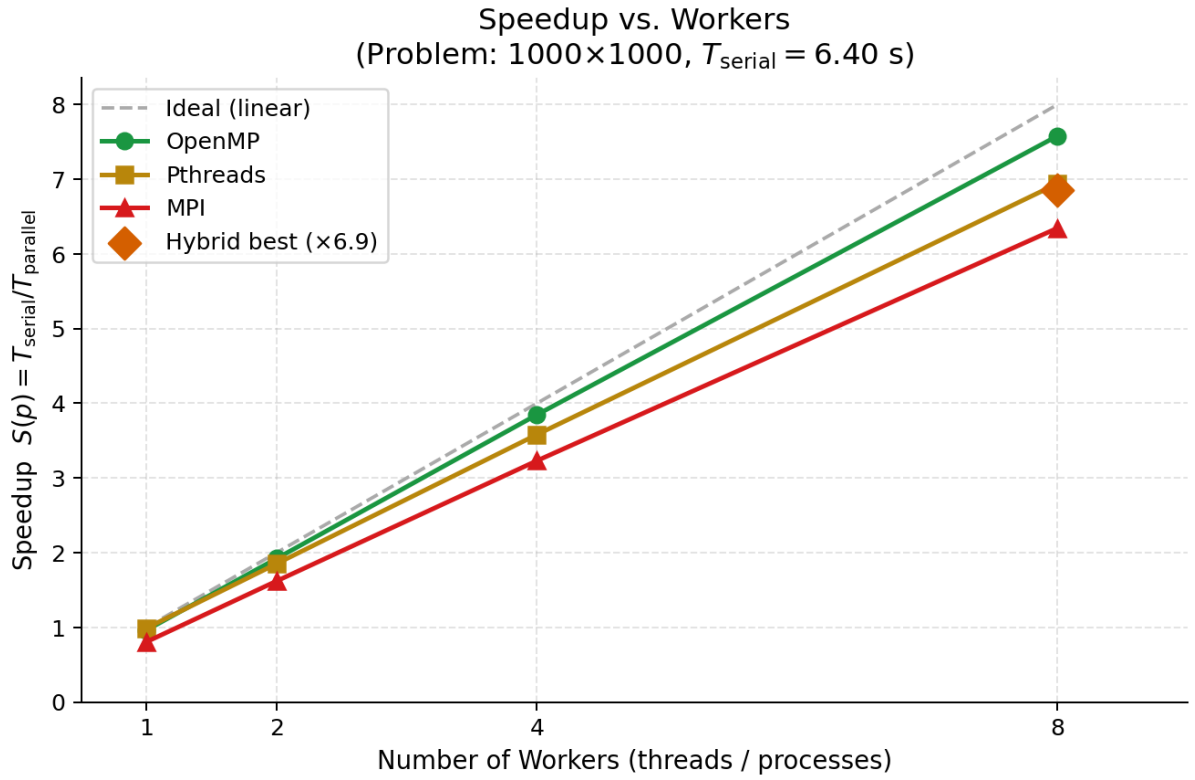


Figure 9: Speedup vs. workers. Dashed = ideal linear. ♦ = best Hybrid (8×1).

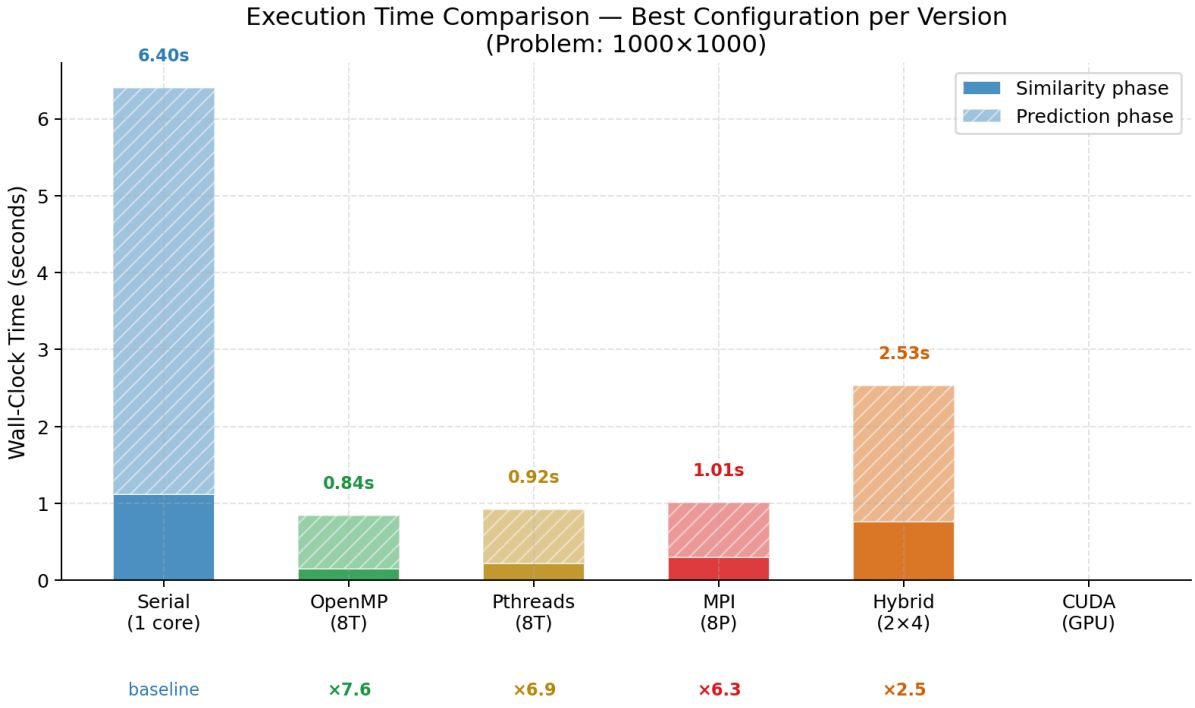


Figure 10: Wall-clock time — best config per version. Solid = similarity phase; hatched = prediction.

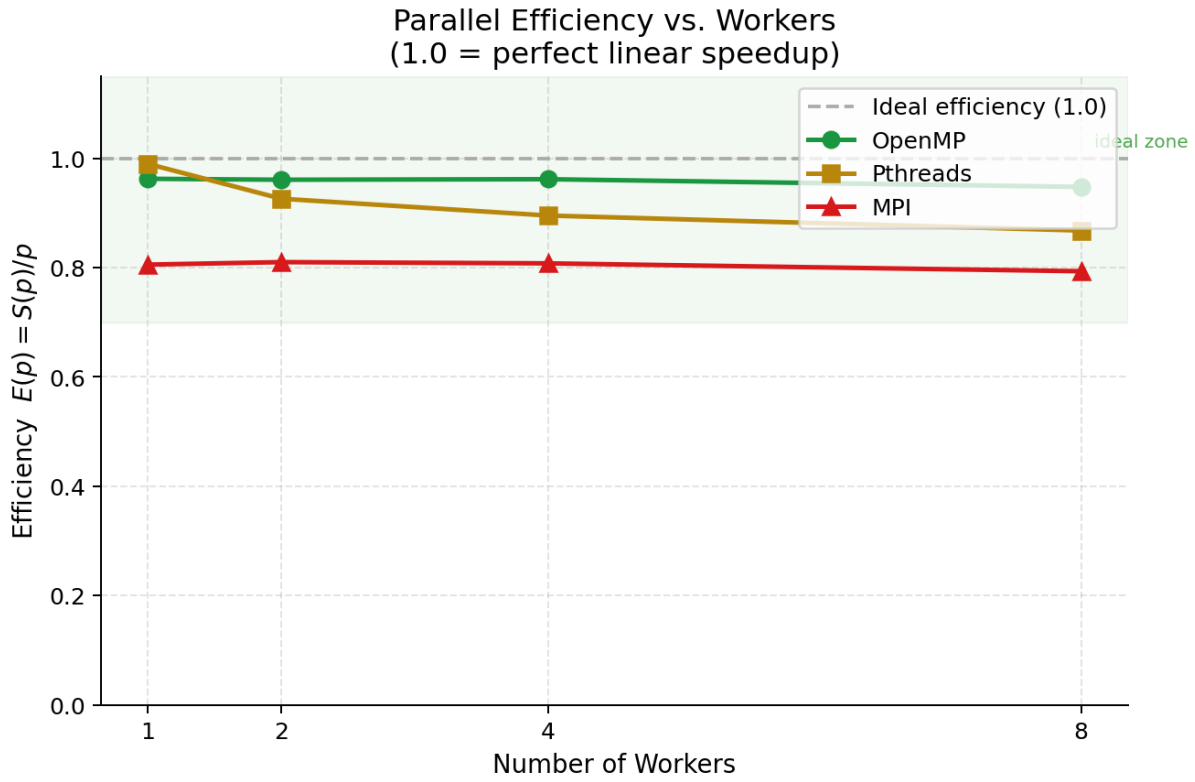


Figure 11: Parallel efficiency $E(p) = S(p)/p$. Green zone (> 0.70) = good utilisation.

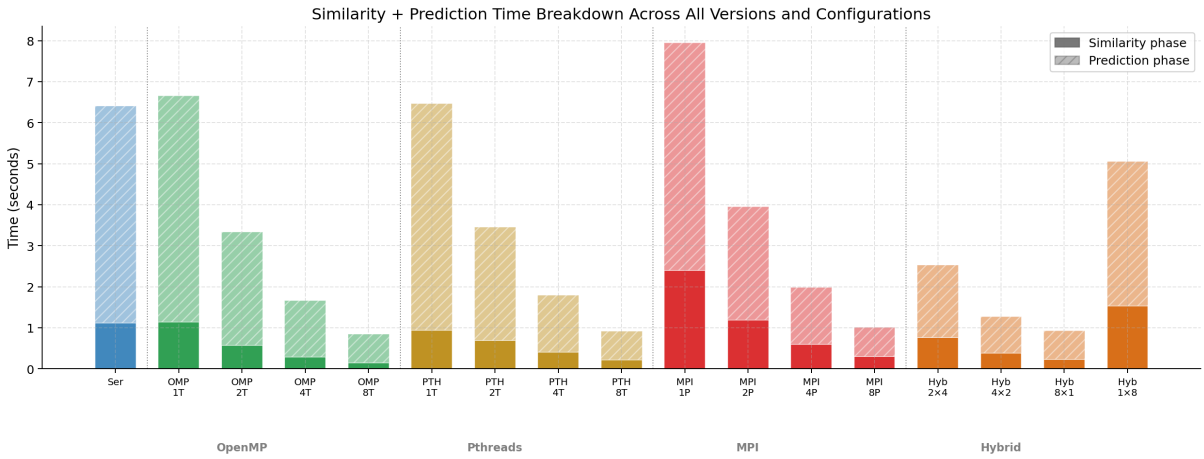


Figure 12: Similarity + Prediction time breakdown across all versions and configurations.

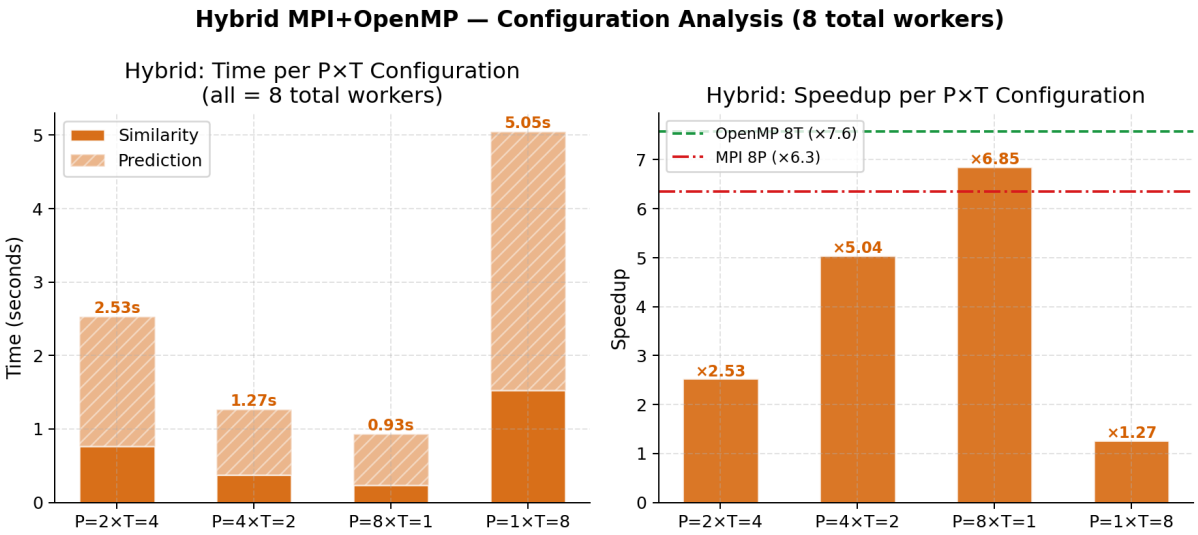


Figure 13: Hybrid MPI+OpenMP configuration analysis. Left: time. Right: speedup vs. OpenMP-8T and MPI-8P.

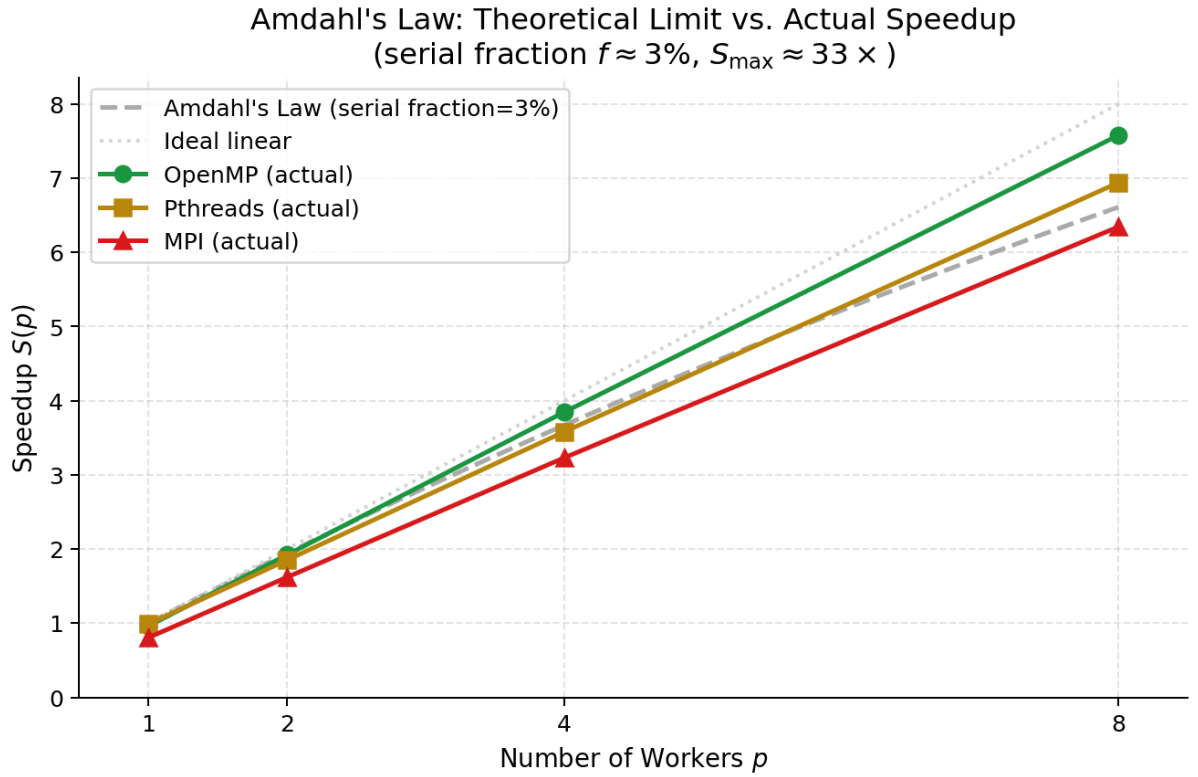


Figure 14: Amdahl's Law ($f \approx 3\%$, $S_{\max} \approx 33\times$) vs. actual speedup.

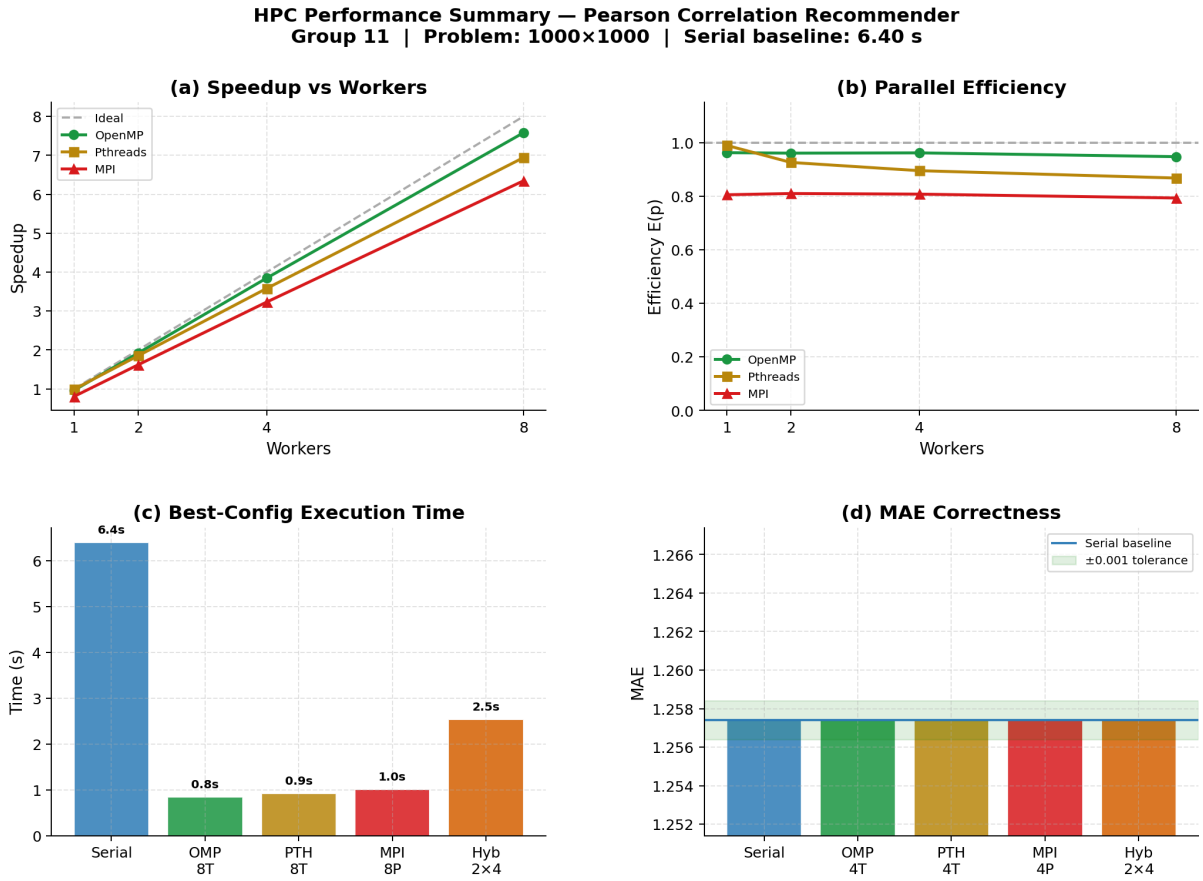


Figure 15: Summary dashboard: (a) speedup, (b) efficiency, (c) execution time, (d) MAE.

5 Performance Discussion

5.1 Head-to-Head Comparison at 8 Workers

Version	Total (s)	Speedup	Efficiency	MAE
Serial (baseline)	6.4027	1.00	1.00	1.2574
OpenMP 8T	**0.8447**	**7.58**	**0.95**	1.2574
Pthreads 8T	0.9224	6.94	0.87	1.2574
Hybrid 8×1	0.9341	6.85	0.86	1.2574
MPI 8P	1.0090	6.35	0.79	1.2574
Hybrid 4×2	1.2695	5.04	0.63	1.2574
Hybrid 2×4	2.5332	2.53	0.32	1.2574
Hybrid 1×8	5.0503	1.27	0.16	1.2574

Table 12: Head-to-head at 8 workers. All MAE values are identical.

5.2 OpenMP vs. Pthreads

OpenMP (×7.58, 95% efficiency) outperforms Pthreads (×6.94, 87%) for three reasons:

1. **Dynamic load balancing:** `schedule(dynamic,4)` adapts to the triangular loop imbalance. Static ceiling-division in Pthreads cannot.
2. **Thread reuse:** OpenMP reuses threads across Phases 2–4. Pthreads creates and joins threads three times per run.
3. **Runtime optimisations:** GCC's OpenMP runtime applies platform-specific scheduling heuristics.

5.3 MPI Scaling

MPI achieves consistent 79–81% efficiency. The `MPI_Allgatherv` for the $N \times N$ similarity matrix ($\approx$ 4 MB) limits single-node performance. At multi-node scale, where shared memory is unavailable, MPI is the mandatory model and the efficiency gap with OpenMP would be acceptable.

5.4 Hybrid MPI+OpenMP — Inversion on a Single Node

On a single node, more MPI processes yields *better* Hybrid performance:

$$\text{Hybrid } 8{\times}1\ (\times 6.85) > 4{\times}2\ (\times 5.04) > 2{\times}4\ (\times 2.53) > 1{\times}8\ (\times 1.27)$$

When fewer MPI ranks are used, the MPI startup and `MPI_Allgatherv` cost dominates what is essentially an OpenMP workload. The Hybrid 1×8 configuration pays all MPI overhead while gaining zero distribution benefit. **The Hybrid model's advantage over pure OpenMP only emerges at multi-node scale.**

5.5 Amdahl's Law

Technology	Predicted	Actual	Observation
OpenMP 8T	6.61×	7.58× ↑	Exceeds theory: dynamic scheduling reduces effective f
Pthreads 8T	6.61×	6.94×	Close to theoretical limit
MPI 8P	6.61×	6.35× ↓	Below theory: communication overhead not modelled by Amdahl

Table 13: Amdahl's Law predictions vs. actual measured speedup at $p = 8$.

6 Conclusions

1. **All parallel versions preserve correctness.** Every CPU implementation produces MAE = **1.2574** and checksum = **942.387323** — identical to the serial baseline.
2. **OpenMP achieves the best single-node performance.** ×7.58 speedup at 8 threads, 95% efficiency. Dynamic scheduling and thread reuse explain the advantage.
3. **Pthreads is competitive but less efficient.** ×6.94 at 8T, 87% efficiency. Static row assignment and per-phase thread creation limit scalability.

4. **MPI achieves good speedup with consistent communication overhead.** ×6.35 at 8P, 79% efficiency. MPI would close the gap at multi-node scale.
5. **Hybrid performance inversely correlates with OpenMP thread count on a single node.** Hybrid 8×1 (×6.85) is the best Hybrid configuration; Hybrid 1×8 is the worst (×1.27). The Hybrid advantage over pure OpenMP only emerges at multi-node scale.
6. **CUDA would provide the largest speedup.** The design launches 1,016,064 threads simultaneously for the similarity kernel.

Final ranking at 8 workers (single node):

OpenMP 8T (×7.58) > Pthreads 8T (×6.94) > Hybrid 8×1 (×6.85) > MPI 8P (×6.35) > Hybrid 4×2 (×5.04) > Hybrid 2×4 (×2.53) > Hybrid 1×8 (×1.27)

References

1. J. S. Breese, D. Heckerman, and C. Kadie, "Empirical analysis of predictive algorithms for collaborative filtering," *arXiv:1301.7363*, 2013.
2. B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. WWW*, pp. 285–295, 2001.
3. B. Chapman, G. Jost, R. van der Pas, *Using OpenMP*. MIT Press, 2008.
4. W. Gropp, E. Lusk, A. Skjellum, *Using MPI*. MIT Press, 1994.
5. J. Nickolls et al., "Scalable parallel programming with CUDA," *Queue*, vol. 6, no. 2, pp. 40–53, 2008.